17

Comparativo: Claude Code × OpenCode × Codex

Toda a trilha foi construída sobre uma tese: o que determina o resultado é o Harness, não a ferramenta. Este material é a prova prática disso — as três principais CLIs de desenvolvimento com IA implementam *os mesmos conceitos* com nomes e sintaxes diferentes. Quem entende o conceito troca de ferramenta em uma tarde; quem decorou comandos recomeça do zero.

A IDEIA CENTRAL DESTE MATERIAL

Os três harnesses respondem às mesmas perguntas: o que o agent sabe, quem executa, o que ele nunca pode fazer, a que ferramentas tem acesso, o que ele lembra. Mudam os arquivos e a sintaxe — não a arquitetura.

Nota de validade: levantamento feito em **agosto de 2026** a partir da documentação oficial das três ferramentas. Este é o material da trilha que envelhece mais rápido — confirme na versão que você usa antes de padronizar decisões de time. Pontos sem confirmação oficial estão sinalizados.

1 Panorama: o mapa de tradução

ELEMENTO	CLAUDE CODE	OPENCODE	CODEX CLI
Arquivo de contexto	`CLAUDE.md`	`AGENTS.md` (fallback: `CLAUDE.md`)	`AGENTS.md` + `AGENTS.override.md`
Configuração	`.claude/settings.json`	`opencode.json(c)`	`config.toml` (TOML)
Skills	`.claude/skills/<n>/SKILL.md`	`.opencode/skills/<n>/SKILL.md`	`.agents/skills/<n>/SKILL.md`
Agents	`.claude/agents/*.md` (Markdown)	`.opencode/agents/*.md` (Markdown)	`.codex/agents/*.toml` (**TOML**)
Comandos	`.claude/commands/*.md`	`.opencode/commands/*.md`	Usar **skills** (prompts descontinuados)

Hooks	Declarativos (JSON) + frontmatter	Só **plugins TS/JS**	Declarativos (`hooks.json`/ TOML)
MCP	`mcpServers` em settings	chave `mcp` no config	`[mcp_servers.<n>]`
Proteção de leitura	`permissions.deny`	`permission` + `.opencodeignore`	Permission Profiles (sandbox)
Memória nativa	—	—	`/memories` (opt-in)

2 Arquivo de contexto

	CLAUDE CODE	OPENCODE	CODEX CLI
Nome	`CLAUDE.md`	`AGENTS.md`	`AGENTS.md`
Global	`~/.claude/CLAUDE.md`	`~/.config/opencode/AGENTS.md`	`~/.codex/AGENTS.md`
Compatibilidade	—	Lê `CLAUDE.md` e `~/.claude/` como fallback (desligável por variável de ambiente)	Nomes alternativos via `project_doc_fallback_filenames`
Escopo por pasta	`.claude/rules/*.md` com `paths:` no frontmatter	`instructions: []` no config (paths, globs, URLs)	AGENTS.md por diretório + `.override.md`
Limite	Não documentado	Não documentado	**32 KiB** (`project_doc_max_bytes`)
`/init`	Sim	Sim	Sim

> Convergência em curso: **AGENTS.md virou o nome comum** entre OpenCode e Codex, e o OpenCode ainda lê o CLAUDE.md como fallback. Na prática, um projeto pode manter um arquivo de contexto que funciona nas três ferramentas — o material 03 previu isso ao tratar o Harness como ativo do projeto, não da ferramenta.

3 Skills

	CLAUDE CODE	OPENCODE	CODEX CLI
Formato	Idêntico nos três: pasta com `SKILL.md`, frontmatter YAML com `name` e `description`, mais arquivos de apoio		
Caminho	`.claude/skills/`	`.opencode/skills/` + lê `.claude/skills/` e `.agents/skills/`	`.agents/skills/` e `~/.agents/skills/`

Invocação	Automática pela description	Tool nativo `skill`, com permissão por glob	`$nome` ou `/skills`; implícita pela description
Criação assistida	Skill de criação de skills	Manual	`$skill-creator` · `$skill-installer`

É o elemento **mais portável** dos três harnesses: o formato de skill é praticamente o mesmo, e OpenCode e Codex convergem no diretório aberto `.agents/skills/`. Uma skill bem escrita roda nas três ferramentas com pouco ou nenhum ajuste.

> **Codex:** muito tutorial ainda indica `.codex/skills/` — está desatualizado. O caminho oficial atual é `.agents/skills/`.

4 Agents e subagents

	CLAUDE CODE	OPENCODE	CODEX CLI
Built-in	Agent geral + subagents por Task tool	Primários: `build`, `plan` · Subagents: `general`, `explore`, `scout`	`default`, `worker`, `explorer`
Definição	Markdown + frontmatter YAML	Markdown + frontmatter YAML	**TOML autônomo**
Restrição de tools	`tools:` no frontmatter	`permission:` granular por glob	Chaves de config + **`sandbox_mode` por agent**
Invocação	Task tool / delegação automática	`@mention` + Task tool (com `permission.task`)	Explícita ou instruída no AGENTS.md
Times	Agent Teams (experimental, off por padrão)	Em desenvolvimento; diferencial: **múltiplos providers no mesmo time**	Sem "teams"; workflows de subagents na mesma sessão

> **O ganho concreto do Codex aqui:** o "reviewer que não pode escrever" (material 10) é obtido no Claude Code com tools restritas + hook de bloqueio; no Codex, basta `sandbox_mode = "read-only"` na definição do agent — a garantia desce ao sistema operacional, não depende de o modelo respeitar a instrução.

5 Comandos personalizados

	CLAUDE CODE	OPENCODE	CODEX CLI
Onde	`.claude/commands/*.md`	`.opencode/commands/*.md` ou chave `command`	`~/.codex/prompts/*.md` **(descontinuado)**

Argumentos	`$ARGUMENTS`, posicionais	`$ARGUMENTS`, `$1..$3`, `!` `cmd` (saída de shell), `@arquivo`	`$1..$9`, `$ARGUMENTS`, nomeados
Por projeto	Sim	Sim	**Não** — só global
Direção atual	Comandos e skills convergiram; skills é o recomendado para workflows novos	Comandos seguem sendo o mecanismo	Migrar para **skills**

Duas das três ferramentas estão empurrando comandos personalizados na direção de **skills**. Se você está começando agora, escreva skills: é o formato mais portável e o que sobrevive melhor às próximas versões.

6 Hooks — a maior divergência

	CLAUDE CODE	OPENCODE	CODEX CLI
Mecanismo	Declarativo em `settings.json` + frontmatter do agent	**Só plugins TS/JS** — não há bloco declarativo	Declarativo em `hooks.json` ou `[hooks]` no TOML
Eventos-chave	`PreToolUse`, `PostToolUse`, `UserPromptSubmit`, `Stop`, `SubagentStop`, `SessionStart/End`, `PreCompact` — e muitos outros	`tool.execute.before/after`, `session.idle`, `permission.asked`, `file.edited`, `todo.updated`…	`PreToolUse`, `PostToolUse`, `PermissionRequest`, `UserPromptSubmit`, `Stop`, `SubagentStart/Stop`, `Pre/PostCompact`, `SessionStart/End`
Bloqueio	`exit 2` + stderr	`throw new Error()` no plugin	`exit 2` ou `permissionDecision: "deny"`
Extras	Hooks por agent no frontmatter	Acesso total à API em TypeScript	**Hooks precisam ser confiados** (hash) · **managed hooks** por admin · `async: true` nativo

▸ COMO ESCOLHER PELO ESTILO DE HOOK

Se o time quer **guardrails declarativos, versionados e legíveis por qualquer um**: Claude Code ou Codex. Se o time quer **lógica arbitrária e integração profunda**, e tem gente confortável com TypeScript: OpenCode entrega mais poder ao custo de manter código.

Dois recursos do Codex que merecem atenção de quem trabalha em empresa: hooks não-gerenciados só rodam depois de revisados e confiados (defesa contra repositório com hook malicioso), e os **managed hooks** definidos pelo admin não podem ser desativados pelo desenvolvedor.

7 Proteção de contexto e segredos

	CLAUDE CODE	OPENCODE	CODEX CLI
Arquivo ignore	`.claudeignore` **não é mecanismo oficial**	`.opencodeignore` existe *(pouco documentado)*	`.codexignore` **não confiável**
Mecanismo oficial	`permissions.deny` em settings.json — ex.: `Read(./.env)`, `Read(./secrets/**)`	Bloco `permission` (allow/ask/deny por glob); `.env` é **deny por padrão**	**Permission Profiles**: deny por path/glob e por domínio de rede, reforçado pelo SO
Natureza	Regra de permissão de ferramenta	Regra de permissão por glob	**Barreira de sandbox**

> **Correção importante para quem seguiu o material 06:** os três ecossistemas migraram de "arquivo de ignore" para **sistemas de permissão**. A regra prática mudou de "liste no ignore" para "negue explicitamente na configuração de permissões". O princípio permanece intacto: **segredo real não fica no repositório**, e nenhum desses mecanismos substitui isso.

8 Sandbox, aprovação e planejamento

	CLAUDE CODE	OPENCODE	CODEX CLI
Modelo de permissão	Regras allow/ask/deny por ferramenta, com modos de permissão	`allow`/`ask`/`deny` por glob; última regra correspondente vence	**Sandbox + approval como camadas independentes**
Isolamento real	No nível da ferramenta	No nível da ferramenta	**No nível do SO** (Seatbelt, bwrap+seccomp, sandbox do Windows)
Planejamento	Plan Mode (Shift+Tab / `/plan`)	Agent primário `plan` (Tab)	`/plan` + `/goal` (objetivo persistente)
Governança corporativa	Settings gerenciados	Managed files + preferências MDM	`requirements.toml` — admin força política, sandbox e hooks

O `/goal` do Codex não tem equivalente direto: define um objetivo **persistente** que sobrevive ao longo da sessão e serve simultaneamente como prompt e como critério de conclusão — parente próximo do gate de fase do PLAN.md (material 07), porém no nível da ferramenta.

9 Memória entre sessões

	CLAUDE CODE	OPENCODE	CODEX CLI

Nativa	Não	Não	**Sim** — `/memories`, opt-in, armazenamento local com redação automática de segredos
Alternativa	RAG do projeto construído com hooks (material 14)		RAG próprio continua útil: versionável e compartilhável com o time

10 O que isso significa na prática

✓ O QUE É PORTÁVEL

Todo o Bloco B: SPEC.md, PLAN.md e o raciocínio de scaffold não dependem de ferramenta nenhuma.

Skills: mesmo formato nas três.

Arquivo de contexto: AGENTS.md funciona em duas, e a terceira lê como fallback.

Conceitos de hook: os eventos têm nomes diferentes, mas o ciclo de vida é o mesmo.

✗ O QUE PRECISA SER REESCRITO

Sintaxe de agents: Markdown vs TOML.

Implementação de hooks: declarativo vs plugin TypeScript.

Regras de permissão: cada uma com sua gramática.

Comandos personalizados: caminho, sintaxe e — no Codex — o próprio mecanismo.

> É a confirmação da tese do material 03: **o Harness é ativo do projeto, não da ferramenta**. Quem escreveu SPEC, PLAN e convenções bem feitas leva 80% do trabalho para qualquer CLI. Quem só decorou onde ficam os arquivos, recomeça.

11 Como escolher

SE O SEU CONTEXTO É…	A ESCOLHA NATURAL
Guardrails declarativos, ecossistema maduro de agents e skills	**Claude Code**
Liberdade de provider, lógica de automação arbitrária, time confortável com TypeScript	**OpenCode**
Exigência de isolamento real, governança corporativa, política imposta por admin	**Codex CLI**

E vale dizer o óbvio que costuma ser esquecido: **as três podem conviver no mesmo repositório**. AGENTS.md + skills em `.agents/skills/` + SPEC.md + PLAN.md formam a base comum; cada ferramenta adiciona a sua camada de agents, hooks e permissões.

▸ **COLA PARA FIXAR**

- Contexto: **CLAUDE.md** (Claude Code) · **AGENTS.md** (OpenCode e Codex, com override por diretório no Codex).
- Agents: **Markdown** nos dois primeiros, **TOML** no Codex — que ainda permite **sandbox por agent**.
- Skills é o elemento **mais portável**: mesmo SKILL.md, e `.agents/skills/` virando padrão comum.
- Hooks: **declarativos** no Claude Code e no Codex; **plugins TypeScript** no OpenCode.
- Arquivo de ignore saiu de cena: hoje o mecanismo é **permissão explícita** — e no Codex, **sandbox reforçada pelo SO**.
- Conceito não muda, sintaxe muda: **o Harness pertence ao projeto**, não à ferramenta.